# Диаграммы компонентов и развертывания

Диаграмма **компонентов** позволяет определить архитектуру разрабатываемой системы, установив зависимости между программными компонентами.

Диаграмма компонентов предоставляет общую картину архитектуры системы, помогает разработчикам и архитекторам лучше понять ее структуру и взаимосвязи, а также является полезным инструментом для коммуникации и документирования архитектурных решений.

Диаграмма компонентов разрабатывается для следующих целей:

- визуализация общей структуры исходного кода программной системы;
- спецификация исполнимого варианта программной системы;
- обеспечение многократного использования отдельных фрагментов программного кода;
- представление концептуальной и физической схем баз данных;

ИЗ ЧЕГО СОСТОИТ?

## Компоненты

Программные части, их роли и зависимости.

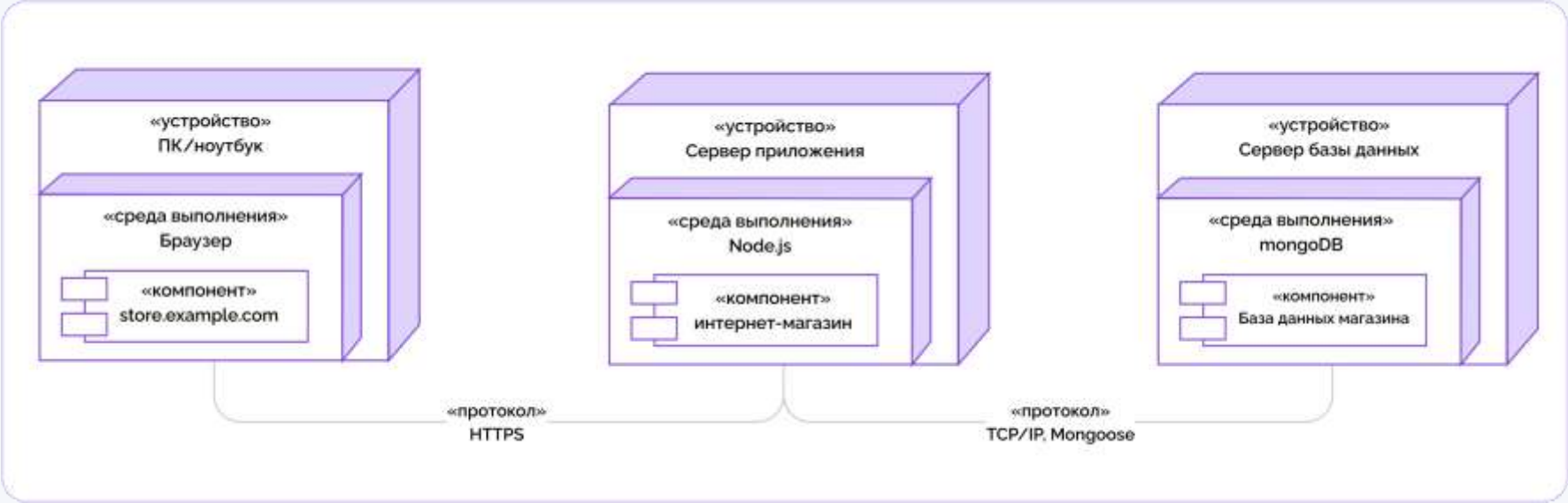
ГДЕ РАБОТАЕТ?

## Развертывание

Устройства, среды выполнения, артефакты и соединения.

# Диаграммы компонентов и развертывания

Диаграмма **развертывания** – это тип UML-диаграммы, которая показывает архитектуру исполнения системы, включая такие узлы, как аппаратные или программные среды исполнения, а также промежуточное программное обеспечение, соединяющее их.
Диаграммы развертывания обычно используются для визуализации физического аппаратного и программного обеспечения системы. Используя его, вы можете понять, как система будет физически развернута на аппаратном обеспечении.



# Берем за основу ваше уже готовое ТЗ

Сквозной пример: веб-приложение «Запись на консультацию»

## Что есть в ТЗ

- Посмотреть свободные слоты
- Записаться на консультацию
- Отменить свою запись

## Какие нужны ответственности

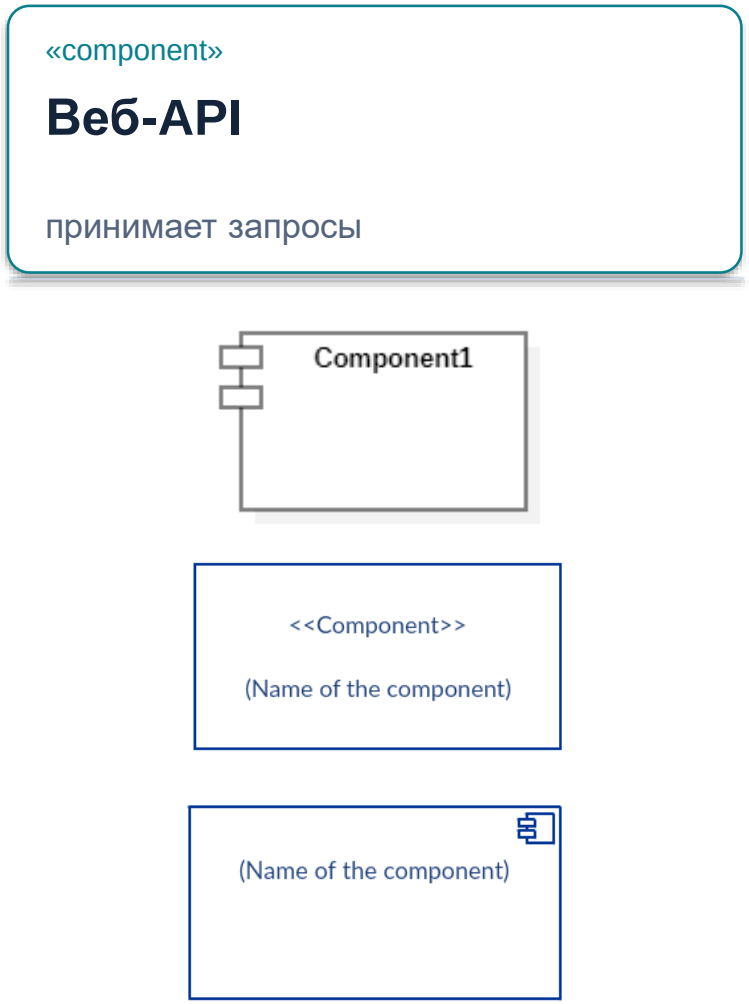
**Веб-интерфейс**
показать и собрать данные

**Веб-API**
принять запрос и вернуть ответ

**Логика записи**
проверить и изменить запись

**Доступ к данным**
прочитать и сохранить данные

# Основные элементы диаграммы компонентов




**Компонент** представляет отдельную часть системы, которая выполняет определенную функцию или имеет определенную роль. Он может быть программным модулем, классом, библиотекой, сервисом, физическим устройством и т. д. Он обычно отображается в виде прямоугольника с его именем или обозначением.

Компонентами могут быть программные компоненты, такие как база данных или пользовательский интерфейс; или аппаратные компоненты, такие как схема, микросхема или устройство; или бизнес-подразделение, такое как поставщик, платежная ведомость или доставка.

# Интерфейсы

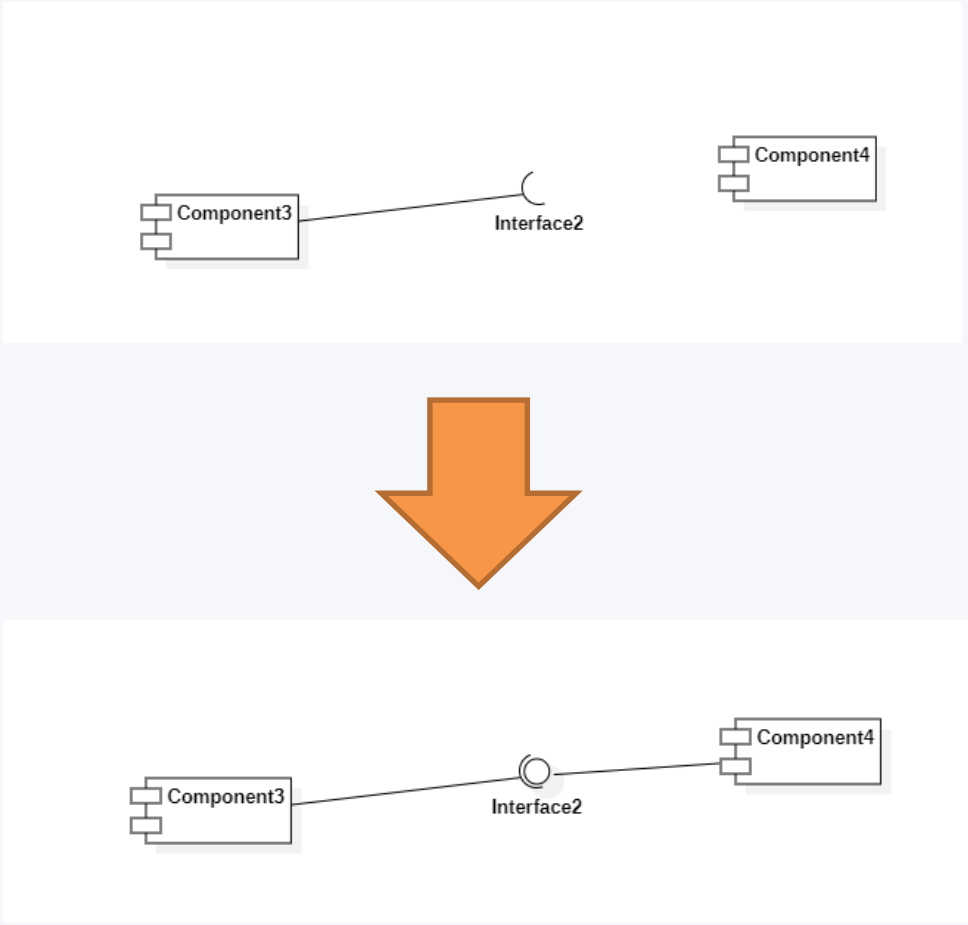



**Предоставляемый интерфейс** на диаграмме компонентов указывает на то, какие возможности или функции предоставляет конкретный компонент для взаимодействия с другими компонентами или системами. Он определяет способ, по которому другие компоненты могут использовать функциональность этого компонента или обмениваться данными с ним. Графически предоставляемый интерфейс отображается на диаграмме компонентов как окружность со сплошной линией.
Коротко: **Это символ для определения интерфейсов, которые предоставляют функции, данные или услуги. Коротко называют «розетка».**

**Требуемый интерфейс** на диаграмме компонентов указывает на интерфейсы, которые компонент ожидает от других компонентов. Он определяет, какие операции, методы или данные должны быть предоставлены другими компонентами, чтобы компонент мог взаимодействовать с ними. Графически требуемый интерфейс отображается на диаграмме компонентов как полукруг со сплошной линией
**Коротко: Он получает услуги, функции или данные извне. Коротко называют «Леденец»**

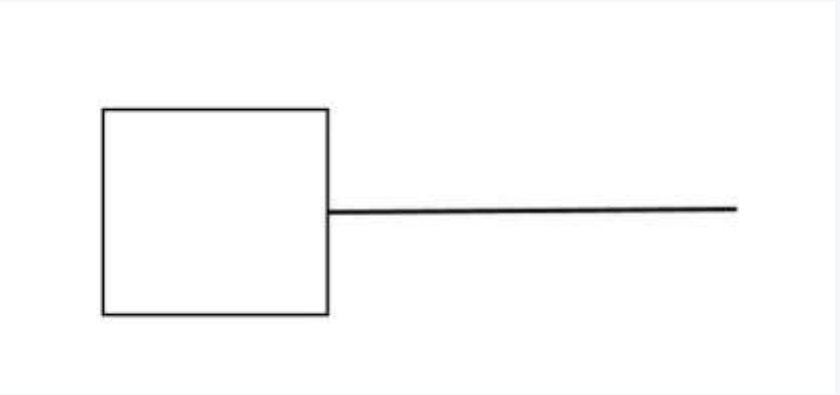
# Связь через интерфейсы

Интерфейсы на компонентных схемах показывают, как компоненты соединены друг с другом и взаимодействуют друг с другом. Соединитель сборки позволяет соединить требуемый интерфейс компонента (представленный полукругом и сплошной линией) с предусмотренным интерфейсом (представленный окружностью и сплошной линией) другого компонента. Это показывает, что один компонент предоставляет услугу, которая требуется другому.

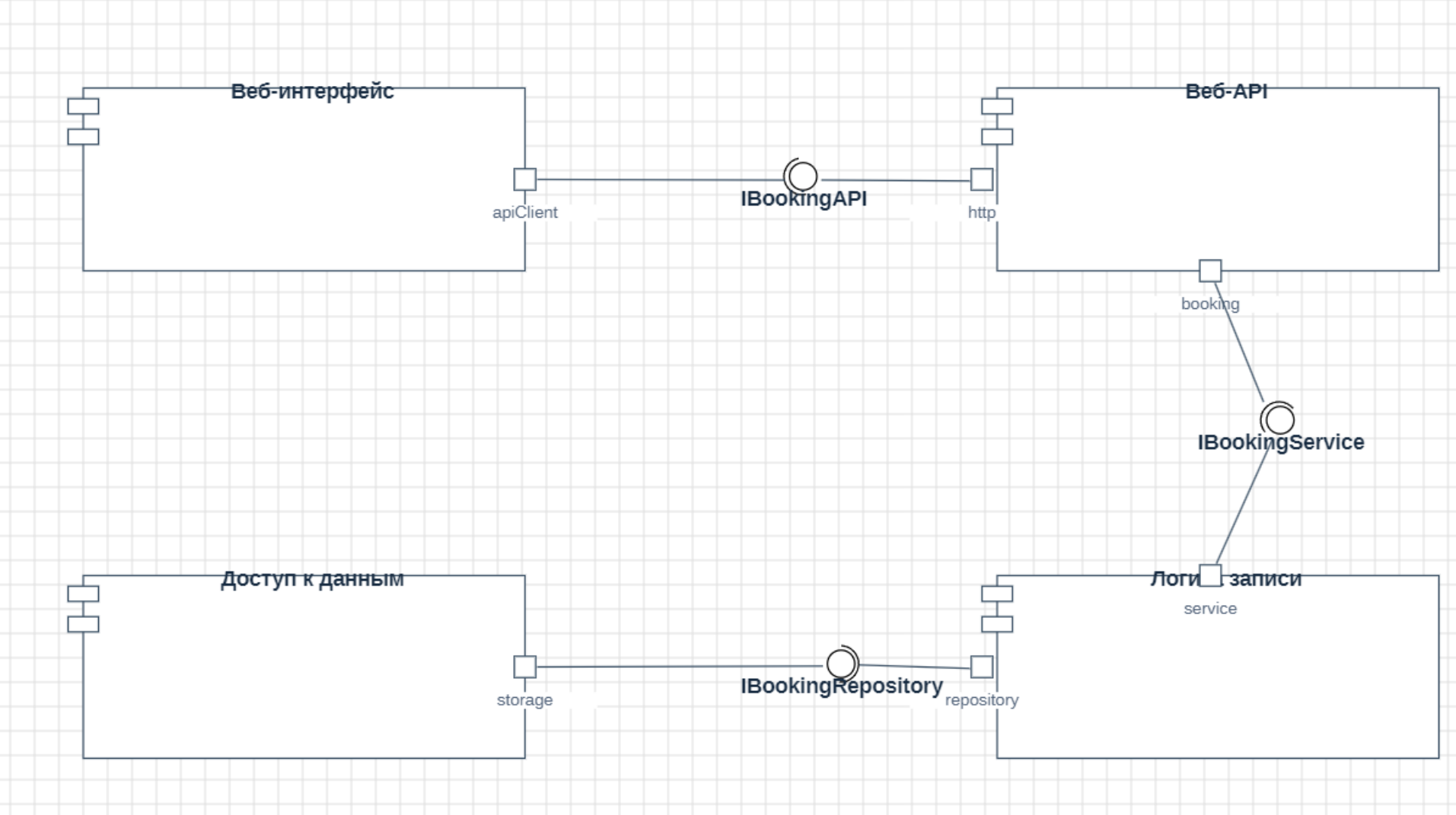


# Порт

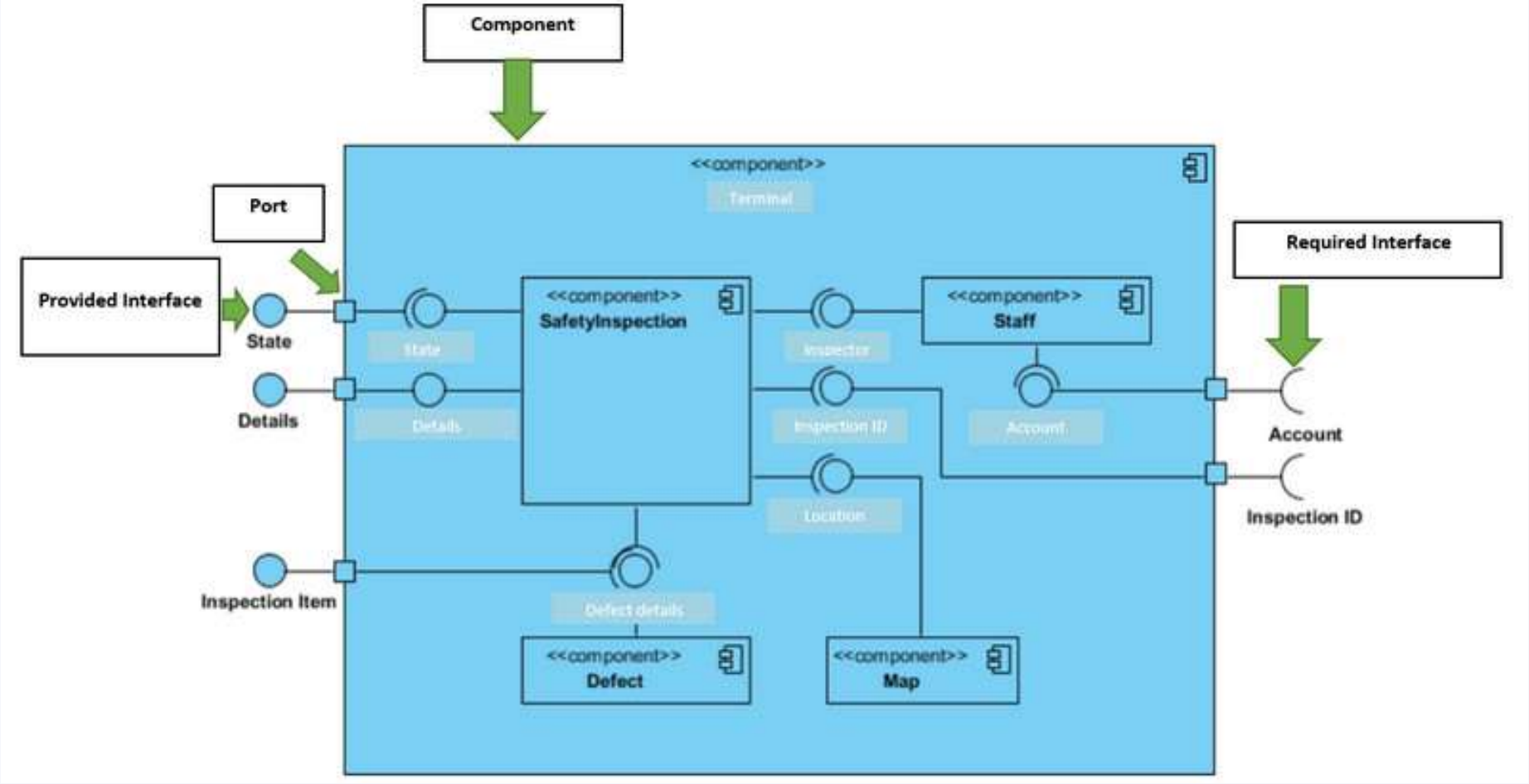
**Порт** представляет точку входа или выхода компонента, через которую он взаимодействует с другими компонентами. Он указывает на интерфейс компонента, через который он предоставляет или получает данные, вызывает методы или осуществляет обмен сообщениями. Графически порт отображается в виде небольших прямоугольников, расположенных на границе компонента. Эти прямоугольники размещаются на разных сторонах компонента, исходя из того, является порт входящим или исходящим.



# Пример 1. Диаграмма компонентов. ИС «Запись на консультацию»

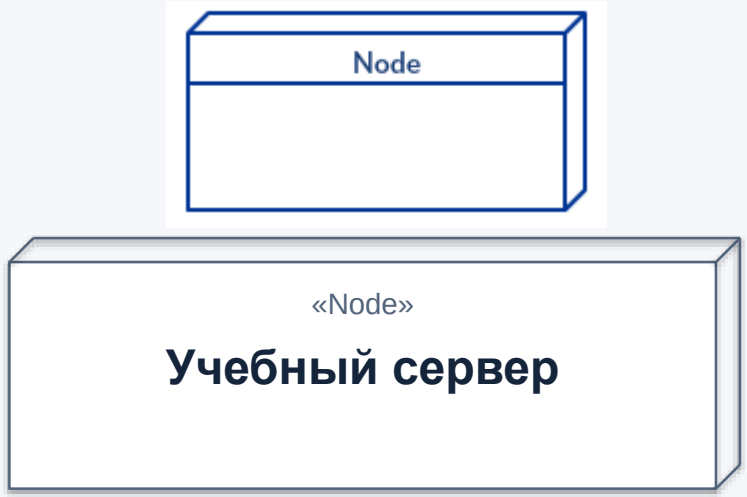


# Пример 2. Диаграмма компонентов



# Основные элементы диаграммы развертывания

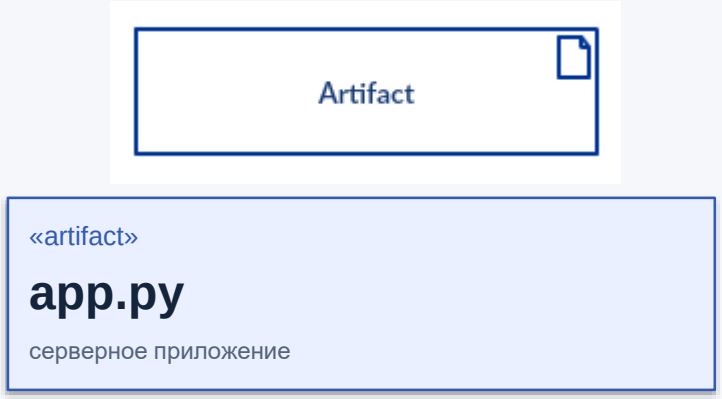
Артефакт - конкретный результат разработки; узел - место его размещения.




Компьютер пользователя, физический сервер.

## Узел

Узел, представленный в виде куба, представляет собой физическую сущность, которая выполняет одну или несколько компонентов, подсистем или исполняемых файлов. Узел может быть аппаратным или программным элементом.




Файл, сборка или база данных внутри узла.

## Артефакт

Артефакты – это конкретные элементы, которые вызваны процессом разработки. Примерами артефактов являются библиотеки, архивы, конфигурационные файлы, исполняемые файлы и т.д.

# Основные элементы диаграммы развертывания



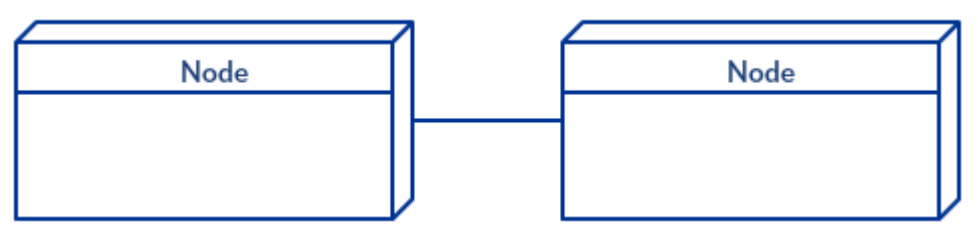

## Среда выполнения

Браузер, Python, PostgreSQL.
Показываем внутри соответствующего устройства.

## Связи

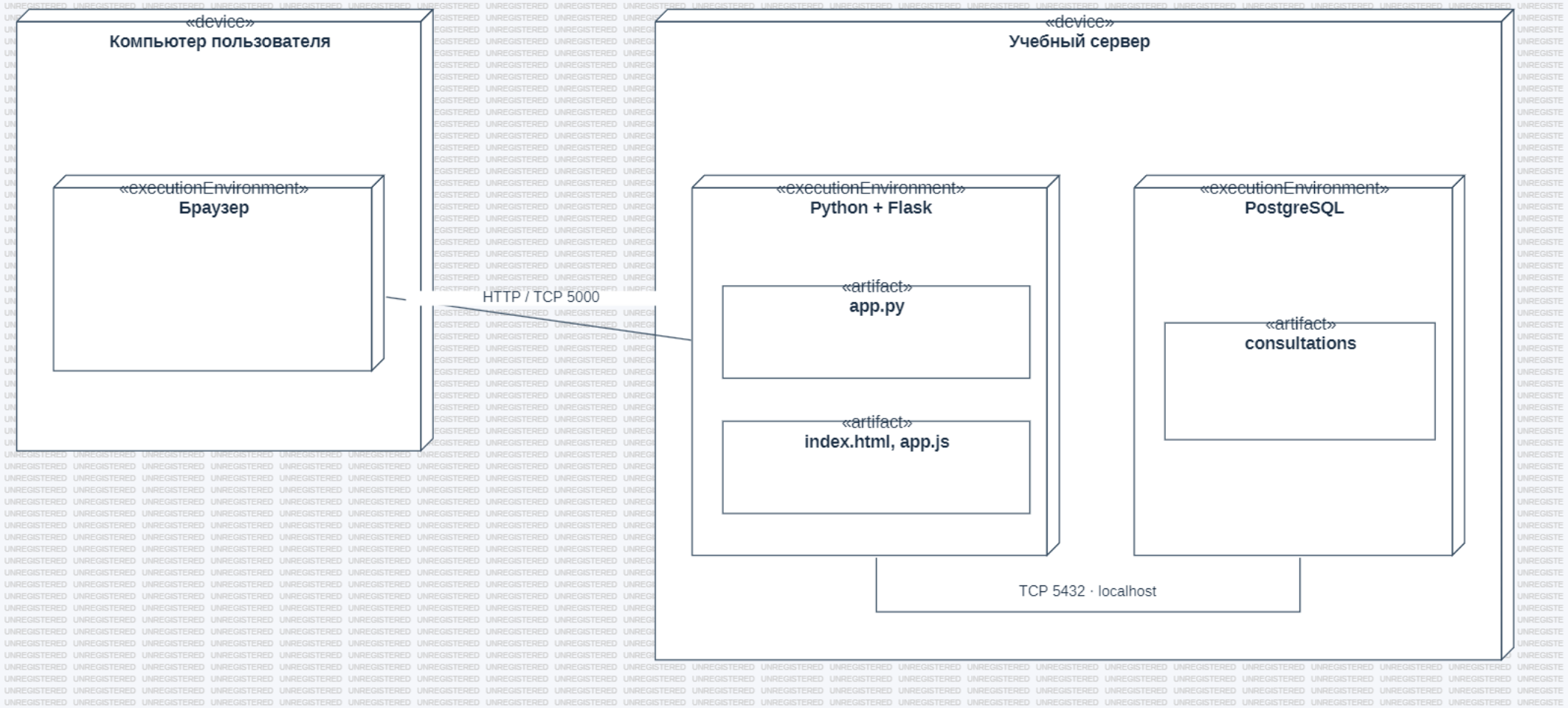
Коммуникационная ассоциация - представлена сплошной линией между двумя узлами. Он показывает путь связи между узлами.

Обычно указывается протокол и/или способ связи

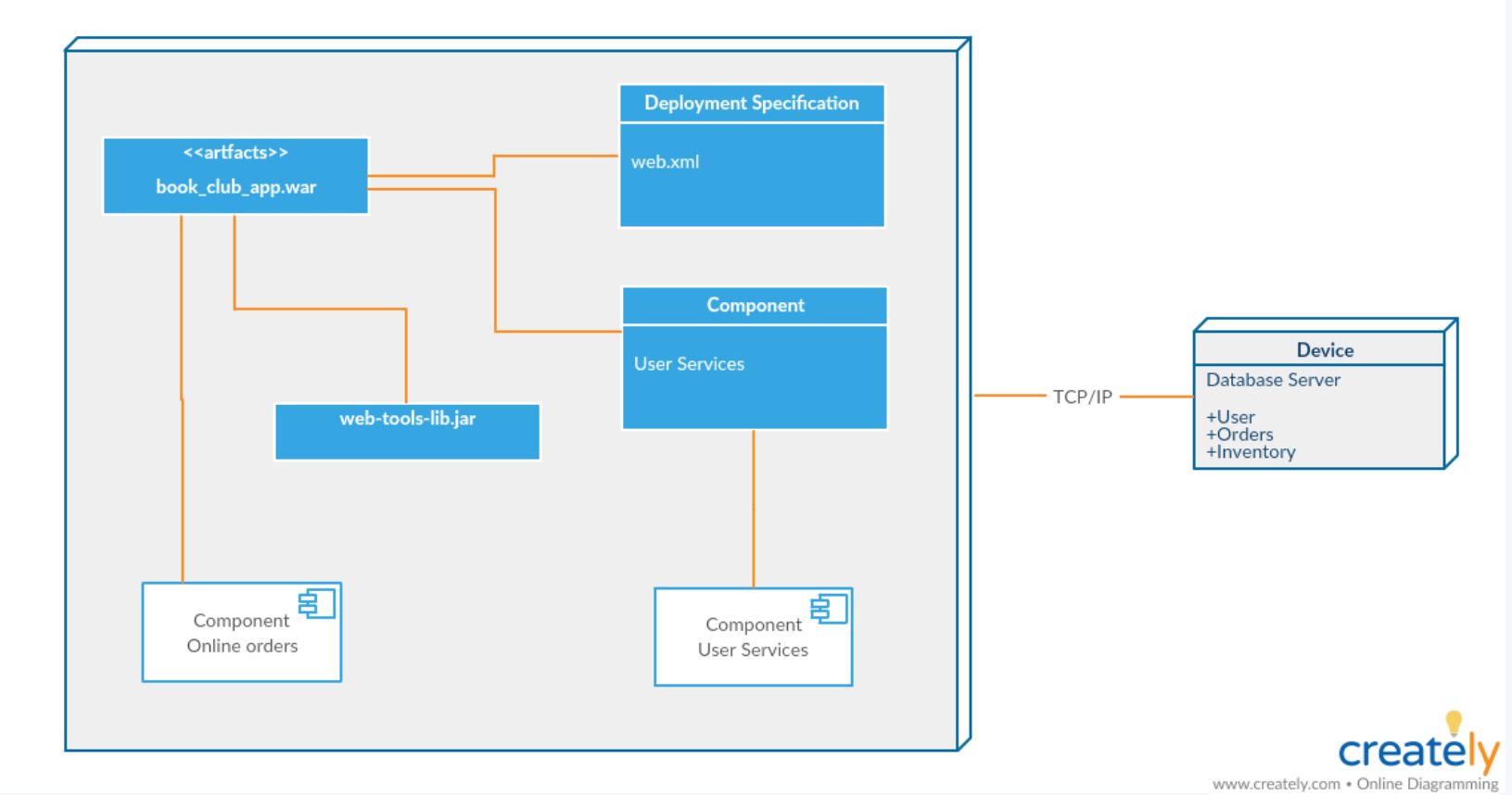


# Пример 1. Диаграмма развертывания

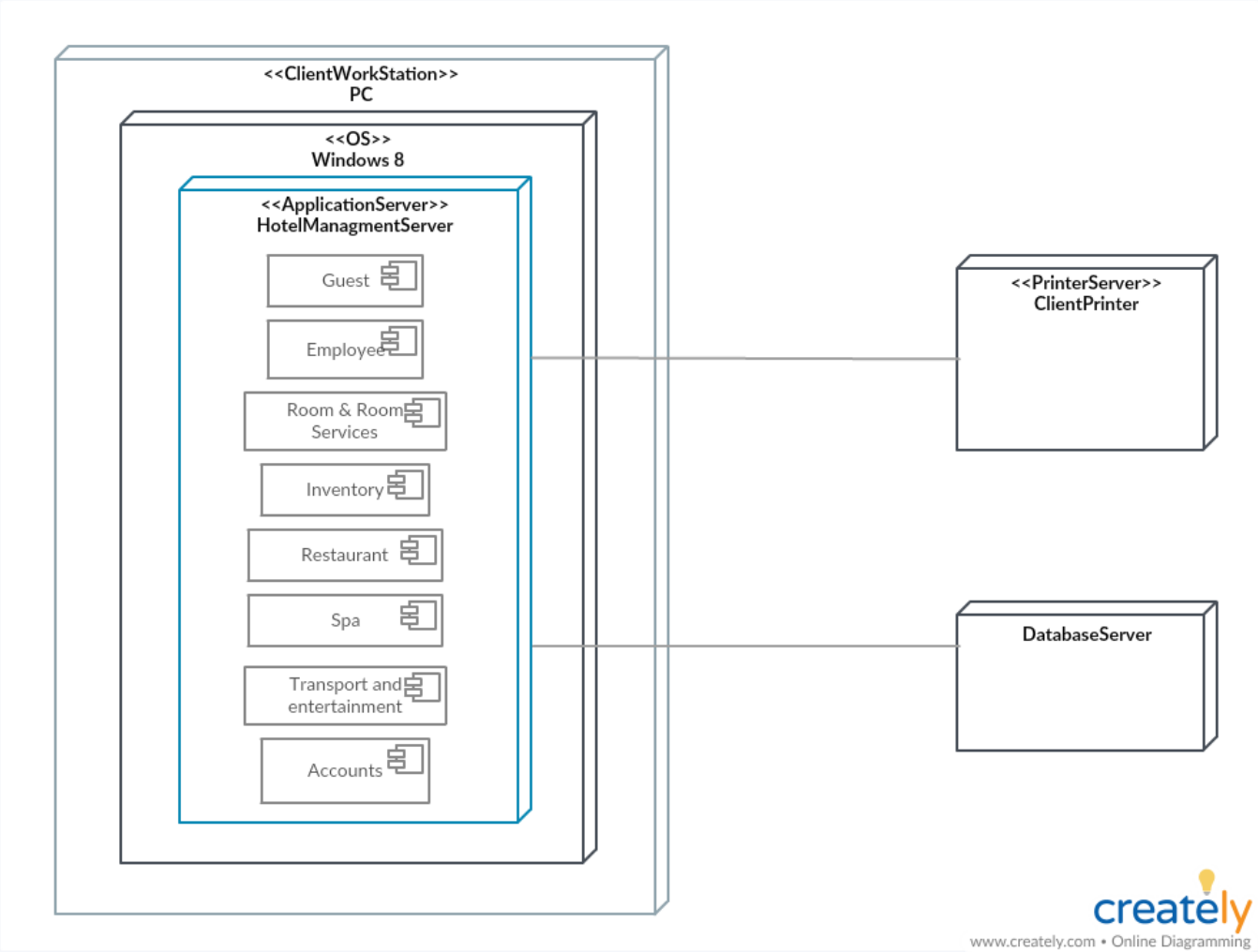
Та же ИС, один учебный сервер; серверное приложение – монолит.



# Пример 2. Диаграмма развертывания



# Пример 3. Диаграмма развертывания



# Как связаны две диаграммы

Компонент → артефакт, который его реализует → место размещения.

| КОМПОНЕНТ | АРТЕФАКТ | ГДЕ РАБОТАЕТ / РАЗМЕЩЕН |
|---|---|---|
| Веб-интерфейс | index.html + app.js | На сервере; копии – в браузере |
| Веб-API<br>Логика записи<br>Доступ к данным | app.py | Python + Flask<br>на учебном сервере |
| Хранение данных | База consultations | PostgreSQL<br>на том же сервере |

# Алгоритм: от вашего ТЗ к двум схемам

01

## Выделите ответственности

Сгруппируйте функции ТЗ в программные части.

02

## Постройте компоненты

Назовите части, покажите и подпишите зависимости.

03

## Определите размещение

Выберите устройства, среды и артефакты.

04

## Соедините и сверьте

Укажите пути связи; проверьте один сценарий из ТЗ.

# Где строить, а также типичные ошибки

**Редакторы**

**Visual Paradigm / draw.io**

**PlantUML/StarUML**

## Типичные ошибки

× Экран или кнопка = компонент

× PostgreSQL = отдельный компьютер

× Зависимость = поток запросов

× Больше блоков = лучше архитектура

# Практика: спроектируйте свою ИС

1

## Диаграмма компонентов

Программные части системы,
их роли и зависимости.

2

## Диаграмма развертывания

Узлы, среды, артефакты
и пути связи.

# Чек-лист перед сдачей

## Компоненты

- [ ] Компоненты соответствуют функциям ТЗ.
- [ ] Понятна ответственность каждой части.
- [ ] Есть «component» или значок компонента.
- [ ] Стрелки зависимостей направлены к используемой части.

## Развертывание

- [ ] Устройства и среды не перепутаны.
- [ ] Артефакты размещены на узлах.
- [ ] Пути связи понятны и подписаны.
- [ ] Две схемы согласованы; допущения обозначены.

Главная проверка: объясните один сценарий из ТЗ по обеим диаграммам.